Name: Gutha Nihitha

Reg.No: 2463021

Class: 5BTCSAIML A

Date: 28/09/2026

CIA-2 Practical Question Paper - Set 1 - Node.js & Asynchronous JavaScript

Attempted Tasks: Task1, Task2, Task3, Task4, Task5, Task6, Task7, Task8, Task9, Task10

Task 1: Command-Line Task Logger — Node.js Development Setup & Running Node.js Files
→ Initialize a new Node.js project using `npm init -y` and create an entry file `logger.js`.
→ Run `logger.js` from the terminal using `node logger.js` and confirm it prints "Task Logger Started".

```
1    console.log("Task Logger Started");
```

OUTPUT:

```
PS C:\Users\nihit\Desktop\LNT_JS> mkdir task-logger


    Directory: C:\Users\nihit\Desktop\LNT_JS


Mode                 LastWriteTime         Length Name
----                 -------------         ------ ----
d-----        28-08-2026     09:25                task-logger


PS C:\Users\nihit\Desktop\LNT_JS> cd task-logger
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> npm init -y
Wrote to C:\Users\nihit\Desktop\LNT_JS\task-logger\package.json:

{
  "name": "task-logger",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC"
}


PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js
```

```
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js
Task Logger Started
PS C:\Users\nihit\Desktop\LNT_JS\task-logger>
```

Task 2: Command-Line Task Logger — Understanding How Node.js Works & Node.js Architecture

→ Add a comment block at the top of `logger.js` explaining, in your own words, how the V8 engine and libuv work together to run your script.

→ Demonstrate non-blocking behaviour by printing a message immediately after triggering `fs.readFile`, showing that it prints before the file's
contents are logged

```
/*
Node.js uses the V8 JavaScript engine to execute JavaScript code.
The libuv library provides the event loop and handles asynchronous
operations such as file-system operations.

When fs.readFile() is called, Node.js does not block the program.
The operation is handled asynchronously, allowing other code to run.
*/

const fs = require("fs");

fs.readFile("tasks.txt", "utf8", (err, data) => {
    if (err) {
        console.log("Error reading file");
        return;
    }

    console.log("File contents:");
    console.log(data);
});

console.log("This message prints before the file contents.");
```

task-logger > tasks.txt

```
1  Complete Node.js assignment
2  Study JavaScript
3  Submit project
```

OUTPUT:

```
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js
This message prints before the file contents.
File contents:
Complete Node.js assignment
Study JavaScript
Submit project
```

Task 3: Command-Line Task Logger — NodeJS Resources & Working with NodeJS Examples
→ Refer to the official Node.js documentation for the `fs` module and list, as a comment, the exact method names you used from it.
→ Adapt one example from the official Node.js docs to read a `tasks.txt` file, noting which doc page you referenced.

task-logger > JS logger.js > ...

```
1   /*
2   Reference:
3   Official Node.js File System documentation:
4   https://nodejs.org/api/fs.html
5
6   Methods used:
7   1. fs.readFile()
8   2. fs.appendFile()
9   */
10
11  const fs = require("fs");
12
13  fs.readFile("tasks.txt", "utf8", (err, data) => {
14      if (err) {
15          console.error("Error reading tasks.txt:", err);
16          return;
17      }
18
19      console.log("Tasks:");
20      console.log(data);
21  });
22
```

OUTPUT:

```
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js
Tasks:
Complete Node.js assignment
Study JavaScript
Submit project
```

Task 4: Command-Line Task Logger — NodeJS REPL Introduction

→ Open the Node.js REPL and test a small snippet that formats today's date using `Date`.

→ Move the working snippet from the REPL into `logger.js` so each logged task is timestamped.

task-logger > JS logger.js > ...

```
1   const fs = require("fs");
2
3   const today = new Date().toLocaleDateString();
4
5   const task = `Task completed on ${today}\n`;
6
7   fs.appendFile("tasks.txt", task, (err) => {
8       if (err) {
9           console.error("Error saving task:", err);
10          return;
11      }
12
13      console.log("Task saved with timestamp:", today);
14  });
```

OUTPUT:

```
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node
Welcome to Node.js v22.21.1.
Type ".help" for more information.
> new Date()
2026-08-28T04:08:20.723Z
> new Date().toLocaleDateString()
'28/8/2026'
> .exit
```

```
> .exit
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js
Task saved with timestamp: 28/8/2026
PS C:\Users\nihit\Desktop\LNT_JS\task-logger>
```

Task 5: Command-Line Task Logger — Node Process Object, Command Line & Terminal I/O
→ Modify `logger.js` to accept a task description as a command-line argument (`process.argv`) and print it.
→ Use `process.stdin` to prompt the user to confirm ("y/n") before saving the task, and print a message based on their response.

task-logger > JS logger.js > ...

```
1   const fs = require("fs");
2
3   const task = process.argv.slice(2).join(" ");
4
5   if (!task) {
6       console.log("Please provide a task.");
7       console.log("Example: node logger.js Complete assignment");
8       process.exit(1);
9   }
10
11  console.log("Task:", task);
12  console.log("Do you want to save this task? (y/n)");
13
14  process.stdin.setEncoding("utf8");
15
16  process.stdin.once("data", (input) => {
17      const answer = input.trim().toLowerCase();
18
19      if (answer === "y") {
20          fs.appendFile("tasks.txt", task + "\n", (err) => {
21              if (err) {
22                  console.error("Error saving task:", err);
```

task-logger > JS logger.js > ...

```
16  process.stdin.once("data", (input) => {
18
19      if (answer === "y") {
20          fs.appendFile("tasks.txt", task + "\n", (err) => {
21              if (err) {
22                  console.error("Error saving task:", err);
23                  return;
24              }
25
26              console.log("Task saved successfully!");
27              process.exit(0);
28          });
29      } else {
30          console.log("Task was not saved.");
31          process.exit(0);
32      }
33  });
```

OUTPUT:

```
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js Complete Node.js ass
ignment
Task: Complete Node.js assignment
Do you want to save this task? (y/n)
y
Task saved successfully!
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js Complete Node.js ass
ignment
Task: Complete Node.js assignment
Do you want to save this task? (y/n)
n
Task was not saved.
PS C:\Users\nihit\Desktop\LNT_JS\task-logger>
```

Task 6: Command-Line Task Logger — Node Packages – NodeMon & Monitoring Applications
→ Install `nodemon` as a dev dependency and add an `npm run dev` script that runs `logger.js` with it.
→ Demonstrate NodeMon automatically restarting the app after you edit and save `logger.js` while it is running.

task-logger > JS logger.js > ...

```
1   const fs = require("fs");
2
3   const task = process.argv.slice(2).join(" ");
4
5   if (!task) {
6       console.log("Please provide a task.");
7       console.log("Example: node logger.js Complete assignment");
8       process.exit(1);
9   }
10
11  console.log("Task:", task);
12  console.log("Do you want to save this task? (y/n)");
13
14  process.stdin.setEncoding("utf8");
15
16  process.stdin.once("data", (input) => {
17      const answer = input.trim().toLowerCase();
18
19      if (answer === "y") {
20          fs.appendFile("tasks.txt", task + "\n", (err) => {
21              if (err) {
22                  console.error("Error saving task:", err);
```

```
19      if (answer === "y") {
20          fs.appendFile("tasks.txt", task + "\n", (err) => {
21              if (err) {
22                  console.error("Error saving task:", err);
23                  return;
24
25              }
26
27              console.log("Task saved successfully!-NodeMon Started");
28              process.exit(0);
29          });
30      } else {
31          console.log("Task was not saved.Please try again.");
32          process.exit(0);
33      }
34  ;
```

OUTPUT:

```
Task: Complete Node.js assignment
Do you want to save this task? (y/n)
[nodemon] restarting due to changes...
[nodemon] starting `node logger.js "Complete Node.js assignment"`
Task: Complete Node.js assignment
Do you want to save this task? (y/n)
y
Task saved successfully!-NodeMon Started
[nodemon] clean exit - waiting for changes before restart
```

Task 7: Command-Line Task Logger — Debugging Node Programs & Debugging Techniques
→ Intentionally introduce a bug (e.g., a typo in a variable name) into `logger.js`, then use `node --inspect` or the VS Code debugger to locate it.
→ Fix the bug and add a short comment describing how you found it.

task-logger > JS logger.js > ...

```
1   const taskName = "Complete Node.js Assignment";
2
3   console.log("Task:", taskname);
```

task-logger > JS logger.js > ...

```
1   const taskName = "Complete Node.js Assignment";
2
3   // Debugging: The error was caused by using taskname instead of t
4   // JavaScript variable names are case-sensitive.
5
6   console.log("Task:", taskName);
```

OUTPUT:

```
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node --inspect-brk logger.js
Debugger listening on ws://127.0.0.1:9229/2bbf282d-d45d-40d3-af5a-33c815344277
For help, see: https://nodejs.org/en/docs/inspector
```

```
C:\Users\nihit\Desktop\LNT_JS\task-logger\logger.js:3
console.log("Task:", taskname);
                     ^

ReferenceError: taskname is not defined
    at Object.<anonymous> (C:\Users\nihit\Desktop\LNT_JS\task-logger\logger.js:
22)
    at Module._compile (node:internal/modules/cjs/loader:1706:14)
    at Object..js (node:internal/modules/cjs/loader:1839:10)
    at Module.load (node:internal/modules/cjs/loader:1441:32)
    at Function._load (node:internal/modules/cjs/loader:1263:12)
    at TracingChannel.traceSync (node:diagnostics_channel:328:14)
    at wrapModuleLoad (node:internal/modules/cjs/loader:237:24)
    at Function.executeUserEntryPoint [as runMain] (node:internal/modules/run_m
n:171:5)
    at node:internal/main/run_main_module:36:49

Node.js v22.21.1
```

Task 8: Command-Line Task Logger — Asynchronous Programming & Callback Functions
→ Write a function `saveTaskCallback(task, callback)` that appends the task to `tasks.txt` using `fs.appendFile` with an error-first callback.
→ Call `saveTaskCallback` and log a success or failure message inside the callback.

task-logger > JS logger.js > ...

```
1   const fs = require("fs");
2
3   function saveTaskCallback(task, callback) {
4       fs.appendFile("tasks.txt", task + "\n", (err) => {
5           if (err) {
6               callback(err);
7               return;
8           }
9
10          callback(null);
11      });
12  }
13
14  const task = "Complete asynchronous Node.js task";
15
16  saveTaskCallback(task, (err) => {
17      if (err) {
18          console.error("Failed to save task:", err.message);
19          return;
20      }
21
22      console.log("Task saved successfully!");
23  });
```

OUTPUT:

Problems 6 Output Terminal ... powershell - task-logger

```
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js
Task saved successfully!
PS C:\Users\nihit\Desktop\LNT_JS\task-logger>
```

Task 9: Command-Line Task Logger — Node Timers & Global Objects
→ Use `setTimeout` to automatically log a "Reminder: review your tasks" message 5 seconds after the app starts.
→ Use `setInterval` to print the number of tasks logged so far every 3 seconds, clearing it with `clearInterval` after 15 seconds.

task-logger > JS logger.js > ...

```
1   const fs = require("fs");
2
3   let taskCount = 0;
4   if (fs.existsSync("tasks.txt")) {
5       const data = fs.readFileSync("tasks.txt", "utf8").trim();
6
7       if (data) {
8           taskCount = data.split("\n").length;
9       }
10  }
11  setTimeout(() => {
12      console.log("Reminder: review your tasks");
13  }, 5000);
14  const intervalId = setInterval(() => {
15      console.log("Number of tasks logged:", taskCount);
16  }, 3000);
17  setTimeout(() => {
18      clearInterval(intervalId);
19      console.log("Task count monitoring stopped.");
20  }, 15000);
```

OUTPUT:

```
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js
Number of tasks logged: 8
Reminder: review your tasks
Number of tasks logged: 8
Number of tasks logged: 8
Number of tasks logged: 8
Task count monitoring stopped.
PS C:\Users\nihit\Desktop\LNT_JS\task-logger>
```

Task 10: Command-Line Task Logger — JavaScript Promises — Introduction, Detail & Revisited

→ Rewrite `saveTaskCallback` from Task 8 as a Promise-based function `saveTaskPromise(task)` using `fs.promises.appendFile`.

→ Chain `.then()` and `.catch()` to log success or failure when calling `saveTaskPromise`.

task-logger > JS logger.js > ...

```
1  const fs = require("fs");
2  function saveTaskPromise(task) {
3      return fs.promises.appendFile("tasks.txt", task + "\n");
4  }
5  const task = "Complete Promise-based Node.js task";
6
7  saveTaskPromise(task)
8      .then(() => {
9          console.log("Task saved successfully!");
10     })
11     .catch((err) => {
12         console.error("Failed to save task:", err.message);
13     });
```

OUTPUT:

```
PS C:\Users\nihit\Desktop\LNT_JS\task-logger> node logger.js
Task saved successfully!
PS C:\Users\nihit\Desktop\LNT_JS\task-logger>
```

CIA-2 Theory Question Paper - Set A - Node.js & Asynchronous JavaScript

Name: Gutha Nihitha Class: 5BTCSAIML A
Reg. No: 2463021 Date: 25/8/2026

Attempted Questions: Q1, Q3, Q4, Q6, Q7, Q8, Q10, Q11, Q13, Q15

Q1.) Explain why installing Node.js also installs npm, and state one command you would run from the terminal to verify both are installed correctly.

Ans - Installing Node.js also installs npm as npm is default packet manager of Node.js.
The command to verify both are installed correctly - node -v.

Q3.) State two reliable resources a developer can use to look up Node.js's built-in module documentation, and explain why using the official docs is preferable to copying code from random blog posts.

Ans — The two reliable resources are
1. Official Node.js API documentation
2. Node.js documentation
Official docs is preferred because it is accurate, maintained and version-specific.

Q4.) In the Node.js REPL, a user types '2+3' and presses Enter, then on the next line types just '_', Explain what is displayed each time and why.

Ans — It displays 5 ~~are~~ as '_' stores the result of previous expression.

Q6.) Explain the specific development problem NodeMon solves, and state the command used to install it as a global package.

Ans — NodeMon automatically restarts a Node.js application whenever a file changes.
command used to install it globally -
npm install -g nodemon.

camlin
Date :

Q7.) Explain ~~the~~ how the '--inspect' flag helps when debugging a Node.js application, and name one tool that can connect to it.

Ans — The '--inspect' flag starts the Node.js application with a debugging interface that allows developers to inspect code, set breakpoints, and monitor variables. A tool that can connect to it is 'Chrome DevTools'.

Q8.) Predict the exact console output, in order, and explain why the order occurs.

```
console.log('A');
setTimeout(() => console.log('B'), 0);
console.log('C');
```

Ans — Output - A
C
B

A, C execute synchronously as there is no delaying of setTimeout() function
B executes later asynchronously as there is a setTimeout() function even though it's '0' secs.

Q10.) predict the exact console output, in order.

```
Promise.resolve(1).then(v => console.log(v)).then(
                    () => console.log(2));
console.log(3);
```

Ans — Output - 3
1
2

3 executes synchronously first, promise .then() callbacks are in a queue so 1 executes first, then only 2 executes.

Name: Gutha Nihitha Class: 5BTCSAIMLA camlin

Reg. No: 2463021 Date: 25/8/2025

Q11.) Rewrite the following promise chain using 'async/await' with proper 'try/catch' error handling.

```
fetchUser()
  .then(u => console.log(u))
  .catch(e => console.error(e));
```

Ans: using async/await

```
async function getUser() {
    try {
        const u = await fetchUser();
        console.log(u);
    } catch (e) {
        console.error(e);
    }
}
```

Q13.) Predict the exact output order and explain which queue (microtask / job queue or macrotask queue) each callback is placed into.

```
console.log('start');
setTimeout(() => console.log('timeout'), 0);
Promise.resolve().then(() => console.log('promise'))
console.log('end');
```

Ans: Output - start
end
promise
resolve

microtask queue is processed before the macrotask queue.

Q15.) Explain the difference between a core module, a local module, and a third-party module in Node.js, giving one concrete example of each.

Ans:- core module - built into Node.js, no installation is needed. E.g:- fs

local module - a module created by the developer within the project.

E.g:- ./math.js

Third party module - A module created by others and installed using npm.

E.g:- express.